

Task1

- 集合：一个容器，可以用来存放一堆数据
- 相同点：都是用来存放多个数据，都可以存放对象
- 不同点：

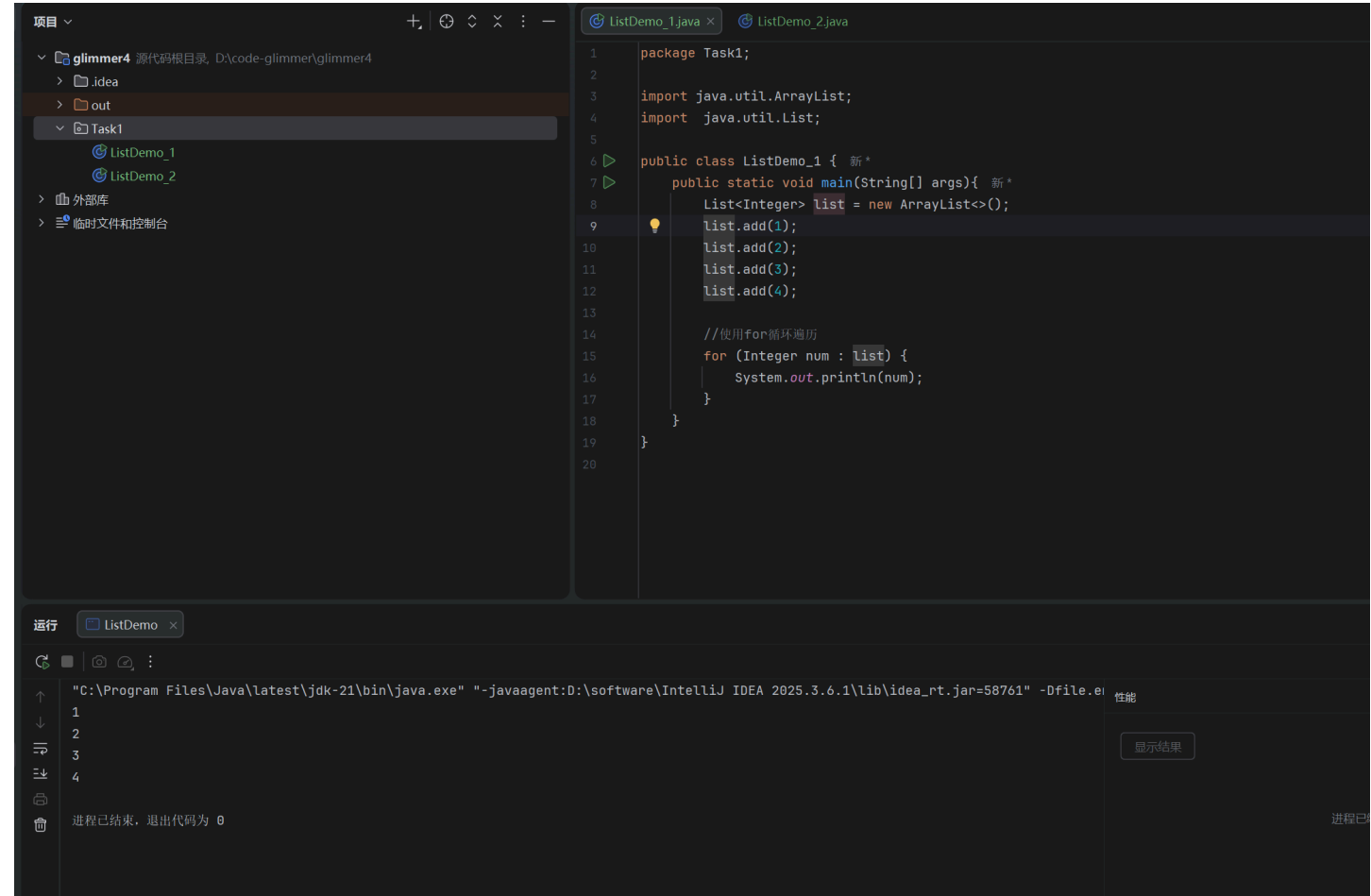
集合	数组
长度可随时改变（add或remove）	长度固定
提供大量现成方法（add、contains）方便操作	功能较少，只有一个length可以查看长度，可用i取对应的元素
只能存对象	可存int等基本类型，也可存对象

数组适合存放数量固定，已知存放个数的数据，但如果数量不固定，需要频繁删减，需要用到集合，另我记得前面的代码中有用到集合，他可以统一的处理一批对象，一体化高效，数组则很零散

Q1:

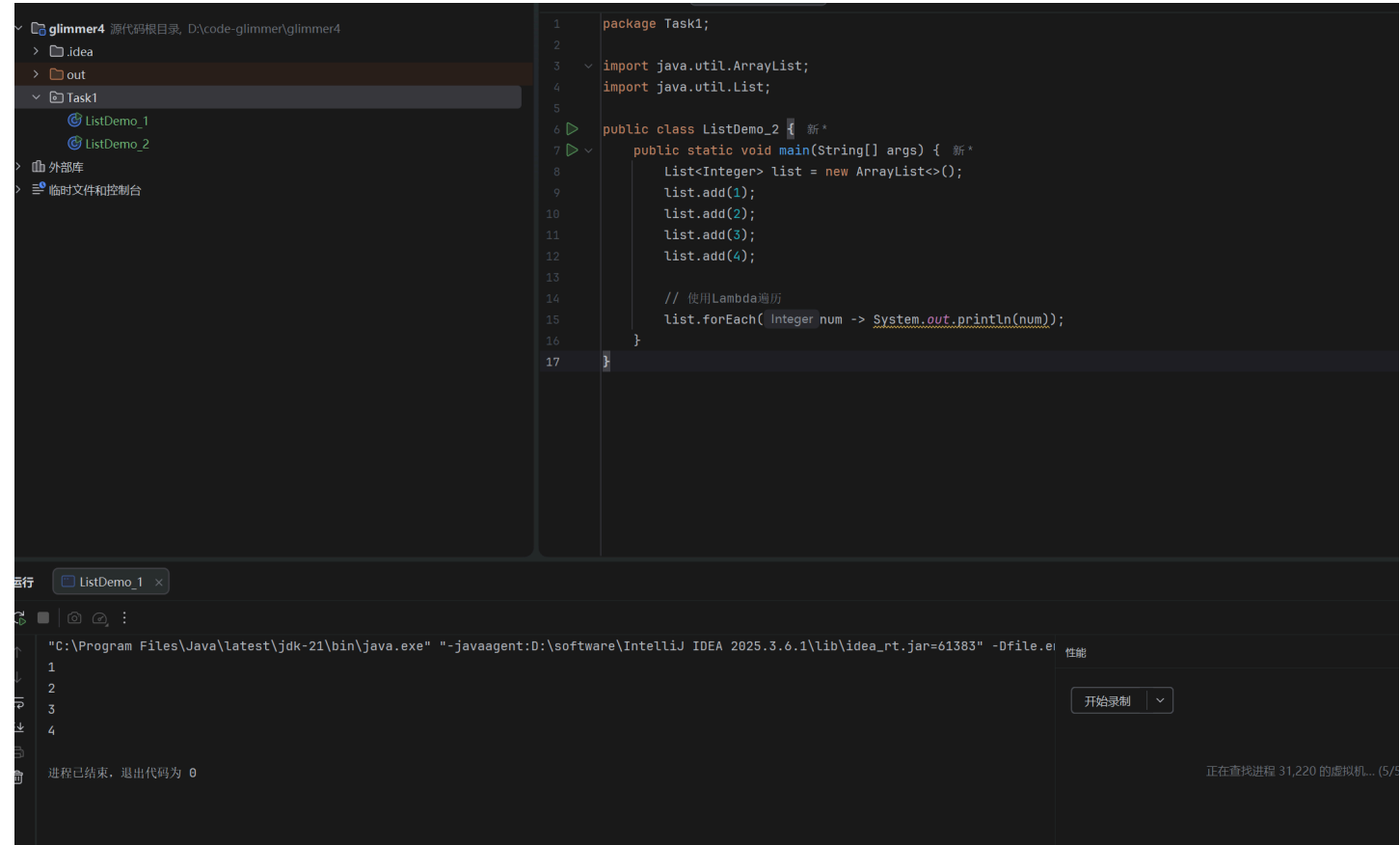
- Collection:单列集合的根接口，他里面装的都是一个个单独的元素以下分出两个子接口
 - List:列表，有序，可重复，像排队
 - ArrayList：底层是数组，查的快，增删慢（想加减一个值，后续所有值要顺次挪位置）
 - LinkedList：底层是链表（每找到一个人他就会告诉你下一个人在哪），查得慢，增删快（只用改增删位相邻两个人的信息指向）
 - Set：集，无序，不可重复，和数学集合一样
 - HashSet：存取速度快，但完全无序
 - TreeSet:会自动按大小排序，但速度稍慢于HashSet
- Map：是所有双列集合的根接口，里面装的是键值对（一个键对应一个值，键类似字典索引，值是真正查到的内容）
 - HashMap：最常用，查找速度极快
 - TreeMap：会根据键自动排序

Q2:



用for循环遍历了list（从list里依次拿出每个元素，把它叫做num）

Q3:



- 匿名内部类：没有名字类，为临时实现某个接口而写，用一次就扔，一般代码很长括号多
- 函数式接口：一个接口里只有一个抽象方法
- lambda表达式：把上面一大段简化，用法是（参数）->{代码}，只有一个参数可省略括号，只有一个代码可省略大括号，比匿名内部类简单得多

- 迭代器遍历：类似导游，带领引导一个个访问集合里的数据
 - 用list.iterator拿到迭代器
 - 用it.hasNext问是否还有下一个
 - 用it.next把下一个取出来

它是遍历最底层最基础的方式，其它for、forEach都是基于此实现的

Task2

（完整版在打包的代码里，就不全截图了）

The screenshot shows an IDE with a project named 'glimmer4'. The left sidebar shows the project structure with folders 'Task1' and 'Task2'. 'Task2' contains files 'Main', 'MyRepository', 'Repository', and 'User'. The main editor displays the code for 'Main.java', which demonstrates the repository pattern for String, User, and Integer types. The code includes saving and retrieving data using the repository interfaces. The bottom panel shows the execution output, which matches the code's logic, displaying the saved data and the retrieved values from the repositories.

```
3 public class Main {
4     public static void main(String[] args) {
5         // 1. 存 String 类型
6         // 创建仓库
7         MyRepository<String> stringRepo = new MyRepository<>();
8         // 存内容
9         stringRepo.save( data: "Hello");
10        stringRepo.save( data: "World");
11        // 分别根据id取出并打印
12        System.out.println("String 仓库: ");
13        System.out.println(stringRepo.getId(0));
14        System.out.println(stringRepo.getId(1));
15
16        // 2. 存 User 类型
17        MyRepository<User> userRepo = new MyRepository<>();
18        userRepo.save(new User( name: "张三", age: 18));
19        userRepo.save(new User( name: "李四", age: 20));
20        System.out.println("User 仓库: ");
21        System.out.println(userRepo.getId(0));
22        System.out.println(userRepo.getId(1));
23
24        // 3. 存 Integer 类型
25        MyRepository<Integer> intRepo = new MyRepository<>();
26        intRepo.save( data: 100);
27        intRepo.save( data: 200);
28        System.out.println("Integer 仓库: ");
29    }
30 }
```

运行结果:

```
String 仓库:
Hello
World
User 仓库:
User{name='张三', age=18}
User{name='李四', age=20}
Integer 仓库:
100
200
```

Task3

```
1 package Task3;
2
3 // 列出角色类型
4 public enum CharacterType { 0个用法 新*
5     SABER, ARCHER, CASTER 0个用法
6 }
```

这里列举均为常量，约定成俗写为大写

